MANIPAL
ACADEMY of HIGHER EDUCATION
(Institution of Eminence Deemed to be University)

MANIPAL SCHOOL OF INFORMATION SCIENCES

(A Constituent unit of MAHE, Manipal)

Linux OS and Scripting

Process_Lab

Registration Number	Name	Branch
261100690021	Praneetha Padmashali	Cybersecurity

Exercise 1: Process identification

1. Display all running processes with their PID, PPID, owner, CPU usage, memory usage, and command.
   ps -eo pid,ppid,user,%cpu,%mem,cmd

2. List only the processes belonging to the current user.
   ps -u "$USER" -f

3. Identify the five processes consuming the most CPU.
   ps -eo pid,ppid,user,%cpu,%mem,cmd --sort=-%cpu | head -n 6

4. Identify the five processes consuming the most memory.
   ps -eo pid,ppid,user,%cpu,%mem,cmd --sort=-%mem | head -n 6

5. Explain why high CPU usage alone does not confirm malicious activity.
   High CPU usage alone does not confirm malicious activity because legitimate applications, software updates, backups, and system tasks may also consume significant CPU resources. A process should be investigated by checking its name, owner, command, parent process, network activity, and behaviour before deciding whether it is malicious.

Exercise 2: Parent and child processes

1. Write a shell script that starts two background processes and displays their PIDs.
   Create a file named process_parent.sh

```
#!/bin/bash

sleep 100 &
PID1=$!

sleep 120 &
PID2=$!

echo "Shell Script PID: $$"
echo "First Child PID: $PID1"
echo "Second Child PID: $PID2"

wait
```

2. Identify the PID and PPID of the shell script.
   ps -eo pid,ppid,user,cmd | grep '[p]rocess_parent.sh'

3. Display the child processes created by the script.

   ps --ppid SCRIPT_PID -f

   Replace SCRIPT_PID with the actual PID of the running script.

4. Explain the purpose of $$, $!, &, and wait.

   $$ - Returns the PID of the current shell.

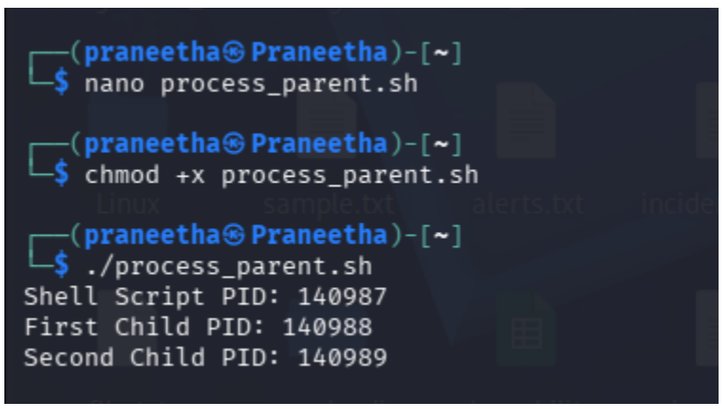
   $! - Returns the PID of the most recently started background process.

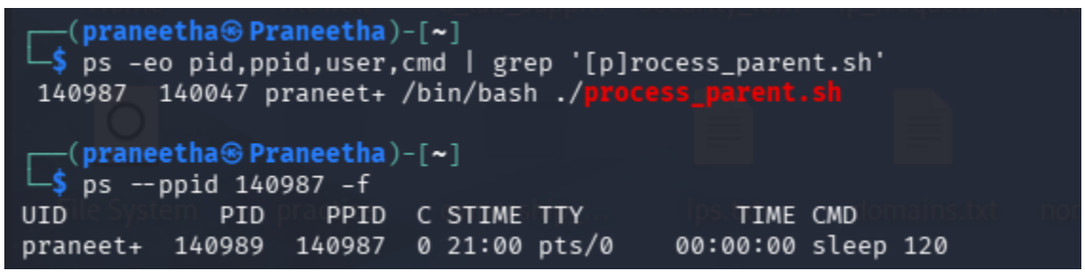
   & - Runs a command in the background.

   wait - Waits for background processes to finish.

5. Observe what happens to the child processes when the parent script terminates.

When the parent script terminates, its child processes may continue running independently. They can become orphan processes and be adopted by a system process such as systemd or another subreaper. The exact behaviour depends on how the script terminates and how the processes are managed.





Exercise 3: Process control using kill

1. Start a background process and record its PID.

   sleep 300 &

   PID=$!

   echo "Process PID: $PID"

2. Suspend the process and verify its state.

   kill -STOP "$PID"

   ps -o pid,stat,cmd -p "$PID"

3. Resume the process and verify its state.

   kill -CONT "$PID"

   ps -o pid,stat,cmd -p "$PID"

4. Terminate the process gracefully.

   kill -TERM "$PID"

5. Verify that the process has terminated.

   ps -p "$PID"

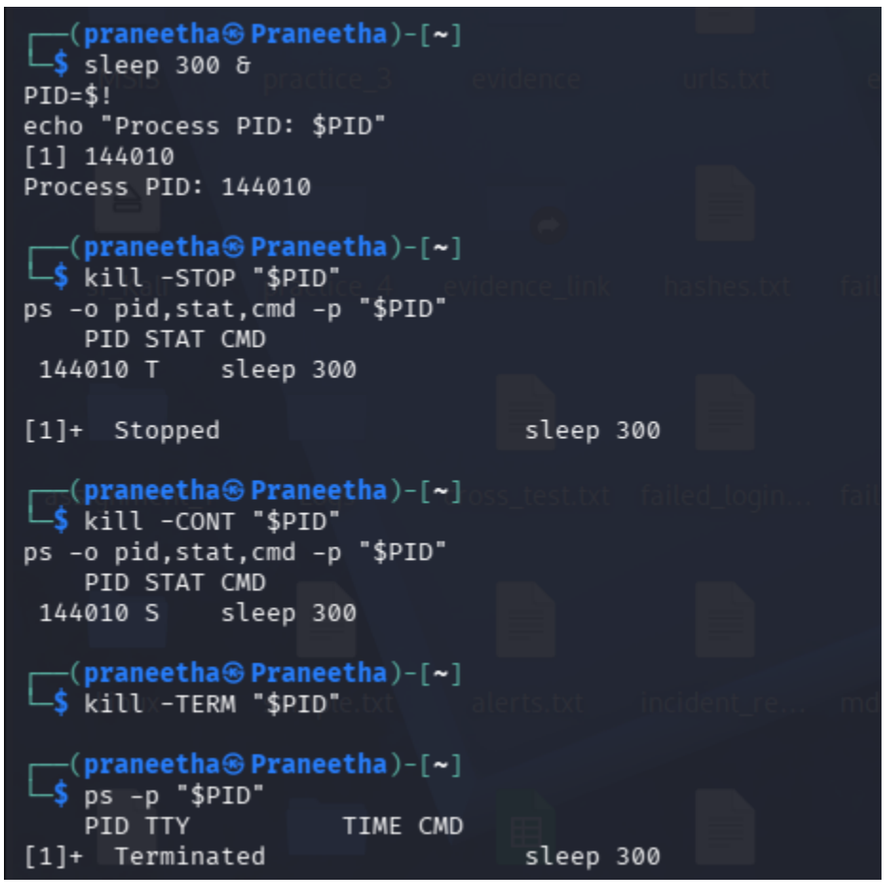
6. Explain the differences between SIGSTOP, SIGCONT, SIGTERM, and SIGKILL.

   SIGSTOP: Suspends a process immediately. It cannot be caught or ignored.

   SIGCONT: Resumes a stopped process.

   SIGTERM: Requests graceful termination. The process can handle or ignore it.

   SIGKILL: Forces immediate termination. It cannot be caught or ignored.



Exercise 4: Graceful and forced termination

1. Write a monitoring script that periodically records a timestamp in a log file. Configure the script to record a shutdown message when it receives SIGTERM.
   Create a file named monitor.sh.

```
#!/bin/bash

LOGFILE="$HOME/monitor.log"

trap 'echo "$(date): Monitoring stopped
gracefully" >> "$LOGFILE"; exit 0' TERM

while true
do
    echo "$(date): Monitoring is running" >>
"$LOGFILE"
    sleep 5
done
```

   ./monitor.sh &

   PID=$!

   echo "$PID"

2. Terminate the script using SIGTERM and inspect the log.
   kill -TERM "$PID"

   sleep 1

   cat "$HOME/monitor.log"

3. Restart the script and terminate it using SIGKILL.
   ./monitor.sh &

   PID=$!

   sleep 2

   kill -KILL "$PID"

   cat "$HOME/monitor.log"
4. Compare the results and explain why they differ.

   When the script receives SIGTERM, its trap executes and writes a shutdown message to the log before exiting. When it receives SIGKILL, the operating system terminates it immediately, without allowing the script to execute its trap. Therefore, the graceful shutdown message is not written during forced termination.

```
┌──(praneetha㉿Praneetha)-[~]
└─$ chmod +x monitor.sh
./monitor.sh &
PID=$!
echo "Monitoring PID: $PID"
[1] 146386
Monitoring PID: 146386

┌──(praneetha㉿Praneetha)-[~]
└─$ cat "$HOME/monitor.log"
Monday 02 November 2026 09:09:30 PM IST: Monitoring is running
Monday 02 November 2026 09:09:35 PM IST: Monitoring is running
Monday 02 November 2026 09:09:40 PM IST: Monitoring is running
Monday 02 November 2026 09:09:46 PM IST: Monitoring is running
Monday 02 November 2026 09:09:51 PM IST: Monitoring is running
Monday 02 November 2026 09:09:56 PM IST: Monitoring is running

┌──(praneetha㉿Praneetha)-[~]
└─$ kill -TERM "$PID"

┌──(praneetha㉿Praneetha)-[~]
└─$ sleep 1
cat "$HOME/monitor.log"
[1]+  Done                    ./monitor.sh
Monday 02 November 2026 09:09:30 PM IST: Monitoring is running
Monday 02 November 2026 09:09:35 PM IST: Monitoring is running
Monday 02 November 2026 09:09:40 PM IST: Monitoring is running
Monday 02 November 2026 09:09:46 PM IST: Monitoring is running
Monday 02 November 2026 09:09:51 PM IST: Monitoring is running
Monday 02 November 2026 09:09:56 PM IST: Monitoring is running
Monday 02 November 2026 09:10:01 PM IST: Monitoring is running
Monday 02 November 2026 09:10:06 PM IST: Monitoring is running
Monday 02 November 2026 09:10:11 PM IST: Monitoring stopped gracefully
```

```
┌──(praneetha㉿Praneetha)-[~]
└─$ ./monitor.sh &
PID=$!
echo "Monitoring PID: $PID"
[1] 148416
Monitoring PID: 148416

┌──(praneetha㉿Praneetha)-[~]
└─$ sleep 6

┌──(praneetha㉿Praneetha)-[~]
└─$ kill -KILL "$PID"

┌──(praneetha㉿Praneetha)-[~]
└─$ sleep 1
cat "$HOME/monitor.log"
[1]+  Killed                  ./monitor.sh
Monday 02 November 2026 09:09:30 PM IST: Monitoring is running
Monday 02 November 2026 09:09:35 PM IST: Monitoring is running
Monday 02 November 2026 09:09:40 PM IST: Monitoring is running
Monday 02 November 2026 09:09:46 PM IST: Monitoring is running
Monday 02 November 2026 09:09:51 PM IST: Monitoring is running
Monday 02 November 2026 09:09:56 PM IST: Monitoring is running
Monday 02 November 2026 09:10:01 PM IST: Monitoring is running
Monday 02 November 2026 09:10:06 PM IST: Monitoring is running
Monday 02 November 2026 09:10:11 PM IST: Monitoring stopped gracefully
Monday 02 November 2026 09:12:56 PM IST: Monitoring is running
Monday 02 November 2026 09:13:01 PM IST: Monitoring is running
Monday 02 November 2026 09:13:06 PM IST: Monitoring is running
Monday 02 November 2026 09:13:11 PM IST: Monitoring is running
Monday 02 November 2026 09:13:16 PM IST: Monitoring is running
```

Exercise 5: Multiple processes using killall

1. Start three instances of a lab program with a unique process name.

Use a harmless lab program named labprocess.

bash -c 'exec -a labprocess sleep 300' &

bash -c 'exec -a labprocess sleep 300' &

bash -c 'exec -a labprocess sleep 300' &

2. Display the PID and owner of each instance.

   ps -eo pid,user,args | grep '[l]abprocess'

3. Terminate one instance using its PID.

   kill -TERM "$PID"

4. Terminate the remaining instances using killall.

   create a separate executable script named labprocess:

   #!/bin/bash

   exec sleep 300

   Run it three times:

   chmod +x ./labprocess

   ./labprocess &

   ./labprocess &

   ./labprocess &

   Display the instances:

   ps -C labprocess -o pid,user,cmd

   Terminate all remaining instances by executable name:

   killall labprocess

5. Verify that all instances have terminated.

   pgrep -a -x labprocess

6. Explain the risk of using killall with a common process name.

Using killall with a common process name can terminate multiple legitimate processes at once. This may interrupt important applications or system services. Therefore, the process name and target processes should be verified before executing the command.

```
┌──(praneetha㉿Praneetha)-[~]
└─$ nano labprocess

┌──(praneetha㉿Praneetha)-[~]
└─$ chmod +x labprocess

┌──(praneetha㉿Praneetha)-[~]
└─$ ./labprocess &
./labprocess &
./labprocess &
[1] 150114
[2] 150115
[3] 150116

┌──(praneetha㉿Praneetha)-[~]
└─$ ps -eo pid,ppid,user,args | grep '[l]abprocess'
 150114  140047 praneet+ labprocess 300
 150115  140047 praneet+ labprocess 300
 150116  140047 praneet+ labprocess 300

┌──(praneetha㉿Praneetha)-[~]
└─$ pkill -f '^sleep 300$'
```

```
┌──(praneetha㉿Praneetha)-[~]
└─$ nano labprocess

┌──(praneetha㉿Praneetha)-[~]
└─$ cp /usr/bin/sleep ./labprocess

┌──(praneetha㉿Praneetha)-[~]
└─$ ./labprocess 300 &
./labprocess 300 &
./labprocess 300 &
[4] 150671
[5] 150676
[6] 150677

┌──(praneetha㉿Praneetha)-[~]
└─$ ps -C labprocess -o pid,ppid,user,stat,cmd
    PID    PPID USER     STAT CMD
 150671  140047 praneet+ S    ./labprocess 300
 150676  140047 praneet+ S    ./labprocess 300
 150677  140047 praneet+ S    ./labprocess 300

┌──(praneetha㉿Praneetha)-[~]
└─$ kill -TERM 150671

┌──(praneetha㉿Praneetha)-[~]
└─$ ps -C labprocess -o pid,ppid,user,stat,cmd
    PID    PPID USER     STAT CMD
 150676  140047 praneet+ S    ./labprocess 300
 150677  140047 praneet+ S    ./labprocess 300
[4]   Terminated                ./labprocess 300

┌──(praneetha㉿Praneetha)-[~]
└─$ killall labprocess

┌──(praneetha㉿Praneetha)-[~]
└─$ pgrep -a -x labprocess
[5]-  Terminated                ./labprocess 300
[6]+  Terminated                ./labprocess 300
```

Exercise 6: Recurring process monitoring using cron

1. Write a shell script that records the current date, time, username, and number of processes owned by the current user.

   Create process_monitor.sh.

```
#!/bin/bash

LOGFILE="$HOME/process_monitor.log"
USERNAME=$(whoami)
COUNT=$(ps -u "$USERNAME" --no-headers | wc -l)

echo "$(date '+%Y-%m-%d %H:%M:%S') | User: $USERNAME | Processes: $COUNT"
>> "$LOGFILE"
```

   chmod +x "$HOME/process_monitor.sh"

   "$HOME/process_monitor.sh"

   cat "$HOME/process_monitor.log"

2. Schedule the script to run every minute.

   Open the current user's crontab:

   crontab -e

   Add the following entry:

   * * * * * /bin/bash "$HOME/process_monitor.sh"

   Save and exit

3. Verify that the scheduled task executes successfully.

   crontab -l

   Wait one or two minutes and inspect the log:

   cat "$HOME/process_monitor.log"

4. Start additional background processes and observe changes in the log.

   sleep 300 &

   sleep 300 &

   sleep 300 &

   tail "$HOME/process_monitor.log"

5. Modify the schedule to run every five minutes.

   crontab -e

   */5 * * * * /bin/bash "$HOME/process_monitor.sh"

6. Remove the exercise entry without deleting other cron entries.

crontab -e

Delete only the line that runs process_monitor.sh, then save and exit.

crontab -l

```
┌──(praneetha㉿Praneetha)-[~]
└─$ nano process_monitor.sh

┌──(praneetha㉿Praneetha)-[~]
└─$ chmod +x process_monitor.sh
./process_monitor.sh

┌──(praneetha㉿Praneetha)-[~]
└─$ cat "$HOME/process_monitor.log"
2026-11-02 21:20:11 | User: praneetha | Processes: 78

┌──(praneetha㉿Praneetha)-[~]
└─$ crontab -e
no crontab for praneetha - using an empty one
Select an editor.  To change later, run select-editor again.
  1. /bin/nano        <---- easiest
  2. /usr/bin/vim.basic
  3. /usr/bin/vim.tiny

Choose 1-3 [1]: 1
crontab: installing new crontab
[1]   Done                    ./labprocess
[2]-  Done                    ./labprocess
[3]+  Done                    ./labprocess
```

```
┌──(praneetha㉿Praneetha)-[~]
└─$ crontab -l
# Edit this file to introduce tasks to be run by cron.
#
# Each task to run has to be defined through a single line
# indicating with different fields when the task will be run
# and what command to run for the task
#
# To define the time you can provide concrete values for
# minute (m), hour (h), day of month (dom), month (mon),
# and day of week (dow) or use '*' in these fields (for 'any').
#
# Notice that tasks will be started based on the cron's system
# daemon's notion of time and timezones.
#
# Output of the crontab jobs (including errors) is sent through
# email to the user the crontab file belongs to (unless redirected).
#
# For example, you can run a backup of all your user accounts
# at 5 a.m every week with:
# 0 5 * * 1 tar -zcf /var/backups/home.tgz /home/
#
# For more information see the manual pages of crontab(5) and cron(8)
#
# m h  dom mon dow   command
* * * * * /bin/bash "$HOME/process_monitor.sh"
```

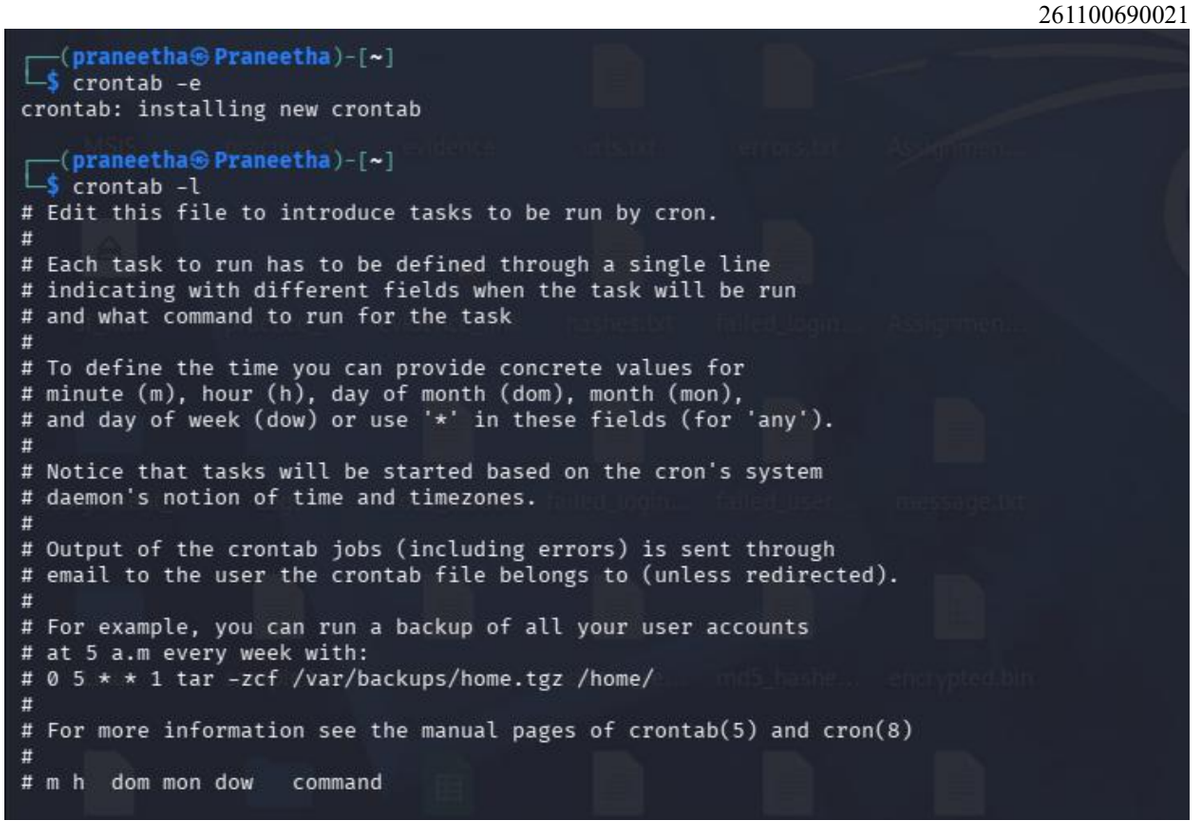
```
┌──(praneetha㉿Praneetha)-[~]
└─$ cat "$HOME/process_monitor.log"
2026-11-02 21:20:11 | User: praneetha | Processes: 78
2026-11-02 21:22:02 | User: praneetha | Processes: 73
2026-11-02 21:23:01 | User: praneetha | Processes: 73
2026-11-02 21:24:01 | User: praneetha | Processes: 72

┌──(praneetha㉿Praneetha)-[~]
└─$ sleep 300 &
sleep 300 &
sleep 300 &
[1] 155514
[2] 155515
[3] 155516

┌──(praneetha㉿Praneetha)-[~]
└─$ tail "$HOME/process_monitor.log"
2026-11-02 21:20:11 | User: praneetha | Processes: 78
2026-11-02 21:22:02 | User: praneetha | Processes: 73
2026-11-02 21:23:01 | User: praneetha | Processes: 73
2026-11-02 21:24:01 | User: praneetha | Processes: 72
2026-11-02 21:25:01 | User: praneetha | Processes: 72
```

```
┌──(praneetha㉿Praneetha)-[~]
└─$ crontab -e
crontab: installing new crontab

┌──(praneetha㉿Praneetha)-[~]
└─$ crontab -l
# Edit this file to introduce tasks to be run by cron.
#
# Each task to run has to be defined through a single line
# indicating with different fields when the task will be run
# and what command to run for the task
#
# To define the time you can provide concrete values for
# minute (m), hour (h), day of month (dom), month (mon),
# and day of week (dow) or use '*' in these fields (for 'any').
#
# Notice that tasks will be started based on the cron's system
# daemon's notion of time and timezones.
#
# Output of the crontab jobs (including errors) is sent through
# email to the user the crontab file belongs to (unless redirected).
#
# For example, you can run a backup of all your user accounts
# at 5 a.m every week with:
# 0 5 * * 1 tar -zcf /var/backups/home.tgz /home/
#
# For more information see the manual pages of crontab(5) and cron(8)
#
# m h  dom mon dow   command
*/5 * * * * /bin/bash "$HOME/process_monitor.sh"
```



Exercise 7: Cron scheduling

Write cron entries for the following requirements:

1. Check failed-login logs every five minutes.

   */5 * * * * /bin/bash "$HOME/check_failed_logins.sh"

2. Capture a process snapshot at the beginning of every hour.

   0 * * * * /bin/bash "$HOME/process_snapshot.sh"

3. Back up security logs daily at 2:30 AM.

   30 2 * * * /bin/bash "$HOME/backup_security_logs.sh"

4. Generate a security report every weekday at 9:00 AM.

   0 9 * * 1-5 /bin/bash "$HOME/security_report.sh"

5. Perform a file-integrity check every Sunday at 3:00 AM.

   0 3 * * 0 /bin/bash "$HOME/file_integrity_check.sh"

6. Run a monitoring script every 15 minutes.

   */15 * * * * /bin/bash "$HOME/monitor.sh"

7. Generate a weekly report every Monday at 8:30 AM.

   30 8 * * 1 /bin/bash "$HOME/weekly_report.sh"

Exercise 8: One-time scheduling using at

1. Schedule a task to record the current date and process list two minutes from now.

   at now + 2 minutes

   At the at> prompt, enter:

   ```
   date > "$HOME/at_output.txt"
   ps -ef >> "$HOME/at_output.txt"
   ```

   Press Ctrl+D

2. Display all pending jobs.

   atq

3. Inspect the commands associated with the scheduled job.

   at -c JOB_NUMBER

   Replace JOB_NUMBER with the actual job number shown by atq.

4. Verify that the output file is created after execution.

   ls -l "$HOME/at_output.txt"

   cat "$HOME/at_output.txt"

5. Explain why the completed job no longer appears in the pending-job list.

   After an at job executes successfully, it is removed from the pending-job queue. Therefore, it no longer appears in the output of atq. Its output file can still remain on disk.

```
┌──(praneetha㉿Praneetha)-[~]
└─$ at now + 2 minutes
warning: commands will be executed using /bin/sh
at Mon Nov  2 21:36:00 2026
at> date > "$HOME/at_output.txt"
at> ps -ef >> "$HOME/at_output.txt"
at> <EOT>
job 1 at Mon Nov  2 21:36:00 2026

┌──(praneetha㉿Praneetha)-[~]
└─$ atq
1       Mon Nov  2 21:36:00 2026 a praneetha

┌──(praneetha㉿Praneetha)-[~]
└─$ at -c 3
Cannot find jobid 3

┌──(praneetha㉿Praneetha)-[~]
└─$ at -c 1
#!/bin/sh
# atrun uid=1000 gid=1000
# mail praneetha 0
umask 2
SESSION_MANAGER=local/Praneetha:@/tmp/.ICE-unix/946,unix/Praneetha:/tmp/.ICE-unix/946; export SESSION_MANAGER
WINDOWID=0; export WINDOWID
QT_ACCESSIBILITY=1; export QT_ACCESSIBILITY
COLORTERM=truecolor; export COLORTERM
XDG_CONFIG_DIRS=/etc/xdg; export XDG_CONFIG_DIRS
XDG_SESSION_PATH=/org/freedesktop/DisplayManager/Session0; export XDG_SESSION_PATH
XDG_MENU_PREFIX=xfce-; export XDG_MENU_PREFIX
```

```
┌──(praneetha㉿Praneetha)-[~]
└─$ ls -l "$HOME/at_output.txt"
-rw-rw-r-- 1 praneetha praneetha 17166 Nov  2 21:36 /home/praneetha/at_output.txt

┌──(praneetha㉿Praneetha)-[~]
└─$ cat "$HOME/at_output.txt"
Monday 02 November 2026 09:36:00 PM IST
UID          PID    PPID  C STIME TTY          TIME CMD
root           1       0  0 13:18 ?        00:00:04 /sbin/init splash
root           2       0  0 13:18 ?        00:00:00 [kthreadd]
root           3       2  0 13:18 ?        00:00:00 [pool_workqueue_release]
root           4       2  0 13:18 ?        00:00:00 [kworker/R-rcu_gp]
root           5       2  0 13:18 ?        00:00:00 [kworker/R-sync_wq]
root           6       2  0 13:18 ?        00:00:00 [kworker/R-kvfree_rcu_reclaim]
root           7       2  0 13:18 ?        00:00:00 [kworker/R-slub_flushwq]
root           8       2  0 13:18 ?        00:00:00 [kworker/R-netns]
root          12       2  0 13:18 ?        00:00:00 [kworker/u8:0-ipv6_addrconf]
root          13       2  0 13:18 ?        00:00:00 [kworker/R-mm_percpu_wq]
root          14       2  0 13:18 ?        00:00:02 [ksoftirqd/0]
root          15       2  0 13:18 ?        00:00:09 [rcu_preempt]
root          16       2  0 13:18 ?        00:00:00 [rcu_exp_par_gp_kthread_worker/0]
root          17       2  0 13:18 ?        00:00:00 [rcu_exp_gp_kthread_worker]
```

Exercise 9: Cancelling scheduled jobs

1. Schedule a harmless task to execute ten minutes from now.

   at now + 10 minutes

   At the at> prompt, enter:

   echo "Scheduled lab task executed" >> "$HOME/at_test.log"

   Press Ctrl+D

2. Identify its job number.

   atq

3. Cancel the scheduled task.

   atrm JOB_NUMBER

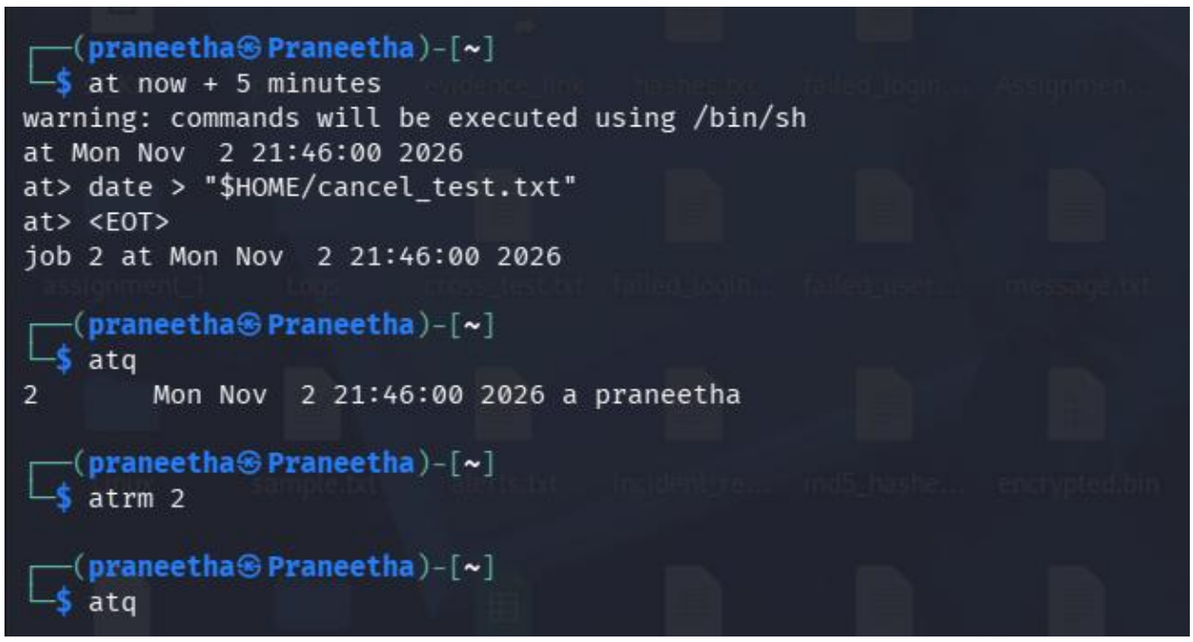
   Replace JOB_NUMBER with the number assigned to your job.

4. Verify that the task has been removed.

atq

5. Explain the difference between cancelling an at job and removing a cron entry.

- atrm cancels a specific pending one-time at job.
- crontab -e allows the user to remove a recurring cron entry.
- An at job is scheduled to execute once, whereas a cron entry can execute repeatedly according to its schedule.
- Removing a cron entry prevents future executions but does not automatically terminate a process that is already running.



Exercise 10: Incident-response scenario

An unfamiliar lab process has multiple running instances and a scheduled task that will restart it.

1. Identify all instances of the lab process and record their PIDs and owners.

   pgrep -a -x labprocess

   ps -C labprocess -o pid,ppid,user,stat,cmd

2. Suspend one instance and verify its state.
   kill -STOP PID
   ps -o pid,ppid,user,stat,cmd -p PID
   Replace PID with the actual process ID.

3. Resume the suspended instance.
   kill -CONT PID
   ps -o pid,stat,cmd -p PID

4. Terminate all instances of the lab process.

   killall -TERM labprocess

   pgrep -a -x labprocess

5. Identify and inspect the scheduled restart task.

   crontab -l

   ls -l /etc/cron.d/

   ls -l /etc/cron.hourly/

   ls -l /etc/cron.daily/

   atq

   at -c JOB_NUMBER

6. Cancel the scheduled restart.

   If the restart was scheduled in the user's crontab, edit it:

   crontab -e

   If it was an at job, cancel it using:

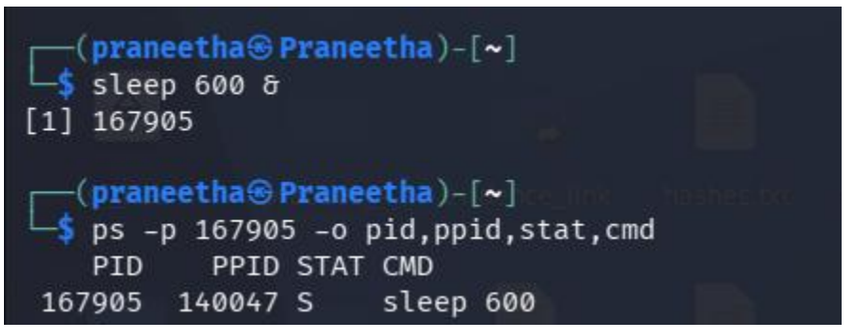
   atrm JOB_NUMBER

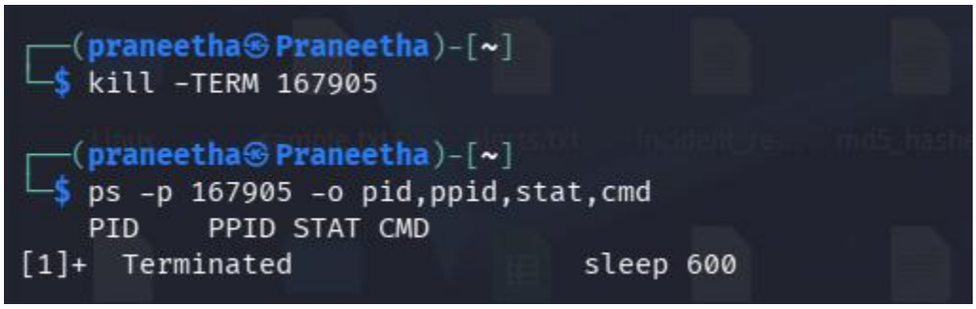
7. Verify that no instances or restart jobs remain.

   pgrep -a -x labprocess

   crontab -l

   atq





8. Incident Report: Unfamiliar Lab Process and Scheduled Restart

1. Objective

To investigate an unfamiliar lab process, identify its running instances, control its execution, inspect scheduled tasks, and verify process termination and job cancellation using Linux process-management commands.

2. Initial Observations

Process name: labprocess

Number of instances identified: 3

PIDs: 150671, 150676, 150677 (as observed during Exercise 5)

Process owner: Current lab user (praneetha)

Process control commands used: kill, killall, SIGSTOP, SIGCONT, and SIGTERM

Scheduling mechanisms examined: Cron and at

Initial process state: The test process in Exercise 10 was running, with state S (interruptible sleep).

3. Actions Performed

- Identified the running labprocess instances and recorded their PIDs.
- Used SIGSTOP to suspend a test process and verified its stopped state using the process status.
- Used SIGCONT to resume the suspended process.
- Sent SIGTERM to terminate a process gracefully.
- Used killall labprocess to terminate the remaining instances of labprocess.
- Inspected cron configuration using crontab -l and removed the process-monitoring cron entry created during the lab exercise.
- Scheduled a one-time task using at, inspected the queued job using atq, and examined its commands using at -c.
- Cancelled the scheduled test job using atrm 2 and verified that the queue was empty.
- Started a test process using sleep 600 &, inspected it using ps, and terminated it using kill -TERM 167905.
- Verified that the test process had terminated.

4. Verification Results

Process termination: Verified for the test process and the labprocess instances targeted during the lab.

Scheduled cron entry removed: Verified for the cron entry created during the exercise.

One-time scheduled job cancelled: Verified; atq showed no remaining jobs after cancellation.

Remaining matching labprocess processes: No matching processes were expected after killall; the final result should be based on the recorded pgrep output.

Reappearance after the expected restart interval: Not verified.

Scheduled restart mechanism removed: The lab-created cron entry and test at job were removed. However, a separate pre-existing restart mechanism for labprocess was not independently confirmed or ruled out.

5. Conclusion

Linux process-management commands were used to identify, suspend, resume, and terminate test processes. Cron and at scheduling mechanisms were also examined. The lab-created cron entry and one-time scheduled job were removed and verified. Although process termination was demonstrated successfully, the absence of a separate, pre-existing mechanism capable of restarting labprocess was not independently established.

6. Recommendations

- Investigate unfamiliar processes before terminating them.
- Examine process ownership, command arguments, and parent processes.
- Inspect cron jobs and other scheduling mechanisms if a process repeatedly reappears.
- Prefer SIGTERM for graceful termination whenever possible.
- Verify process termination and scheduling changes after incident-response actions.
- Document actual PIDs, process owners, command output, and verification results for audit purposes.